

コマ 16: Python 版総合演習・mycc.py 統合

今日のゴール

第 15 回までに作った Python 版 C サブセットコンパイラを `final/mycc.py` に統合し、標準トラックの成果物として完成させる。

この回では新しい構文は追加しない。目的は、これまでの実装を 1 つにまとめ、テストし、読めるコードに整えることである。

```
python3 scaffold/test_runner.py
```

このコマンドで `final/mycc.py` と `final/tests/` を使った最終確認を行う。

1 この回で行うこと

作業	内容
統合	第 15 回までの完成版を <code>final/mycc.py</code> に反映する
総合テスト	<code>final/tests/</code> の 15 本を実行する
バグ修正	落ちたテストを小さい入力に分解して直す
コードレビュー	関数の責務、変数名、コメント、重複を確認する
仕上げ	動作確認方法と実装上の工夫を説明できる状態にする

2 第 1 回から第 15 回までの到達点

ここまでで扱った主な機能は次の通りである。

範囲	機能
式	整数、四則演算、比較、論理、ビット演算

範囲	機能
変数	ローカル変数、代入、スコープ、グローバル変数
制御	if、while、for、break、continue
関数	関数定義、関数呼び出し、再帰、引数、戻り値
型	int、char、ポインタ、配列、構造体
メモリ	lvalue/rvalue、&、*、sizeof、malloc
入出力	文字列リテラル、printf
分割	#include、#define、複数 .c ファイル

第 16 回の作業は、これらが同じ `mycc.py` の中で同時に動くことを確認する総合演習である。

3 final/mycc.py への統合

通常回では `sessions/NN_xxx/mycc.py` を編集してきた。最終成果物は `final/mycc.py` に置く。

```
workbook/
├── sessions/15_preprocess_multifile/mycc.py  # 直前回までの作業ファイル
└── final/mycc.py                          # 最終成果物の統合先
```

まず、第 15 回までの完成版を `final/mycc.py` に反映する。その後、次のコマンドを実行する。

```
python3 scaffold/test_runner.py
```

省略形の `test_runner.py` は、デフォルトで `final/mycc.py` と `final/tests/` を使う。

明示的に指定する場合は次の通り。

```
python3 scaffold/test_runner.py --compiler final/mycc.py --tests final/tests
```

4 final/tests の位置づけ

final/tests/ には、標準トラック到達の参考指標となる 15 本のテストが入っている。

#	テスト	主な確認内容
1	f01_arith.c	算術式
2	f02_vars.c	ローカル変数
3	f03_if.c	条件分岐
4	f04_while.c	while
5	f05_for.c	for
6	f06_break.c	break
7	f07_continue.c	continue
8	f08_func.c	関数呼び出し
9	f09_recur.c	再帰
10	f10_ptr.c	ポインタ
11	f11_array.c	配列
12	f12_struct.c	構造体
13	f13_global.c	グローバル変数
14	f14_string.c	文字列と printf
15	f15_define.c	#define

この 15 本が全通することを標準トラック完成の強い目安とする。

5 テストが落ちたときの切り分け

総合テストで落ちた場合、いきなり大きな入力を読むのではなく、原因を小さく分ける。

1. どのテストが落ちたかを見る
2. そのテストが使っている機能を確認する

3. 該当するセッションの小さいテストを再実行する
4. 入力 C をさらに小さくする
5. 生成アセンブリを見る
6. 必要なら `qemu / gdb` で実行時の値を追う

たとえば `f12_struct.c` が落ちた場合、まず第 12 回の `dot_access.c` と `arrow_access.c` が通るか確認する。第 12 回の小さいテストが通らないなら、構造体メンバのオフセット計算や `codegen_lval()` を見る。第 12 回のテストは通るが `f12_struct.c` だけ落ちるなら、関数引数やポインタ渡しとの組み合わせを疑う。

6 コードレビュー観点

動くだけでなく、読めるコンパイラにする。仕上げとして次の点を確認する。

観点	確認すること
AST 走査	<code>self.codegen()</code> メソッドと <code>self.codegen_lval()</code> メソッドの責務が分かっているか
型	<code>self._type_of_expr()</code> と <code>self._type_of_lval()</code> の役割が明確か
変数管理	<code>self._locals</code> と <code>self._globals</code> の探索順序が明確か
スタック	フレームサイズが 16 バイト境界に揃っているか
関数呼び出し	引数を <code>a0</code> から順に渡しているか
データ領域	文字列、 <code>.data</code> 、 <code>.bss</code> の出力順が整理されているか
エラー	未定義変数や未対応ノードで分かるエラーを出しているか
コメント	複雑な処理だけに短いコメントがあるか

7 完成チェックリスト

- `python3 scaffold/test_runner.py` が通る
- 生成アセンブリを 1 つ以上読んで、どこで何をしているか説明できる

- `self.codegen()`、`self.codegen_lval()`、`self._type_of_expr()`、`self.gen_func()` の役割を説明できる
- 自分が苦労したバグと、その直し方を説明できる
- `final/mycc.py` が最終成果物であることを確認している

8 第 17 回以降

第 17 回以降は進度別に分岐する。

標準トラックの受講者は、Python 版の補修、テスト追加、発表準備を行う。上位トラックを目指す受講者は、Python 版を参照しながら C 版コンパイラへの移植に進む。

移植では、Python の `dict` を C の連結リストへ、Python のクラスを C の `struct` へ写すことが中心になる。

9 テスト

```
python3 scaffold/test_runner.py
```